

NCF Campaign — Measuring the DELIVERED Network: Footprint, Packaging, and Ray-Path Connectivity (task for wavenet_junior)

Hi Chris,

Same structure as the last two memos: TL;DR, your actual task, then end-notes with the evidence and every formula explained. Read the first two, use the third as you go.

This is still the NCF real-waveform workstream (BlueHive3). It has nothing to do with the FTAN/U-Net ML pipeline.

Up to now your job on this campaign has been to watch it and flag problems, and you've been doing that. This is the next step up: a real piece of analysis with a result I want from you, not a status report. I've built and run the whole thing once already so you have a reference answer to check yourself against — the point is not to discover whether the code works, it's for you to run it, read it, and tell me what it means.

TL;DR

- Everything we have ever plotted about this network — the station map, the ray-path heatmap, the 9,743-pair file — describes the network we PLANNED to build, from a data-centre availability inventory. Nobody has ever measured the network we actually DELIVERED. Those are different things, and assuming they were the same is exactly what caused the v1 bug that cost us 954 stations.
- I wrote `metadata3/measure_delivered_network.py` to measure it, and ran it. Headline: 1,023 of 1,999 stations delivered, 510,501 days of real data, 502.1 GB — but only **7,575 usable station pairs** out of 522,753 geometrically possible. Detail: End-notes A.
- The binding constraint is NOT geometry, it's TIME. Two stations can only be correlated over days they were BOTH recording, and most of what we've delivered so far are short deployments — median 229 days each. Detail: End-notes B.
- That number will improve a lot, and I can show why: 96% of the stations still in flight have 5+ years of data, versus 28% of what's landed. The long-running permanent stations are exactly the ones still being processed. Detail: End-notes B.
- Good news on where our losses fell: $\psi = 0.48$, meaning the stations we lost were concentrated in already-dense regions rather than in sparse ones. That is the cheap failure mode, not the expensive one. Detail: End-notes D.
- The backup gap is closed — terravibranium's mirror now holds all 1,023 completed stations (468 GB), up from 1,014. Detail: End-notes F.

YOUR TASK

Run the analysis yourself, then tell me what it means. Two deliverables at the end: a stage doc (`docs/ncf_pipeline_stages/stage_5_delivered_network_coverage.md`, I've already created the skeleton with the fields to fill) and a short Slack post for the team following `docs/memos/_template_slack_research_update.md`.

STEP 1 — set up the analysis environment on axon-1 (one time only)

axon-1 has no scientific Python at all, so make yourself a private virtual environment. This does not touch anything shared and you can delete it any time:

```
/usr/local/bin/python3.11 -m venv ~/venv-wavenet-analysis
~/venv-wavenet-analysis/bin/pip install -r ~/wavenet-epicAI/src/wavenet_pipeline/00_waveform_acquisition/metadata3/requirements-analysis.txt
```

Everything installs as a prebuilt wheel, so nothing compiles and it takes about a minute.

STEP 2 — get the data summary off BlueHive3

The analysis does NOT read the 500 GB of waveform files. It reads one small summary file I produce on BH3 and copy down – about 240 kB. First produce it (this is a job you submit; it takes about two minutes):

```
ssh bluehive3
cd /scratch/tolugboj_lab/delivered_network_analysis
sbatch scan.slurm
```

Check it finished with `squeue -u tolugboj`. Then from axon-1, pull the summary down:

```
cd ~/wavenet-epicAI/src/wavenet_pipeline/00_waveform_acquisition/metadata3
scp -q "bluehive3:/scratch/tolugboj_lab/delivered_network_analysis/cache/*"
delivered_cache/
```

STEP 3 — run the analysis on axon-1

```
cd ~/wavenet-epicAI/src/wavenet_pipeline/00_waveform_acquisition/metadata3
~/venv-wavenet-analysis/bin/python measure_delivered_network.py \
    --stages 2,3 --cache-dir ./delivered_cache --out-dir ./my_run
```

Takes about a minute and writes 16 figures plus CSVs into `my\_run/`. Because it works off the small summary file, you can re-run it as often as you like with different settings and it costs you nothing – that is the point of splitting it this way.

STEP 4 — compare your run against mine

My reference output is committed at `metadata3/delivered\_network\_reference/`. Your `SUMMARY.txt` should match it closely. It will not match EXACTLY, and that is expected: the campaign is still running, so if you re-do STEP 2 you'll pick up stations that have completed since. If it differs wildly, something is wrong – tell me rather than writing it up.

STEP 5 — what I actually want you to answer

Read the figures and answer these four. Numbers alone aren't the answer; I want the reasoning.

1. Look at ``fig15_restriction_tradeoff.png``. If we only allow station pairs closer than 1,110 km we get good resolution (186 km) over 18.5% of the globe. If we allow any distance we get 100% of the globe at 831 km resolution. Which would you choose for a global study, and why? There is no single right answer – I want your reasoning.
2. Look at ``fig13_map_quality_classes.png``. The resolved cells are mostly over continents and the poorly-resolved ones mostly over ocean basins. Explain why that happens. (Think about where our stations physically are, and what directions a path crossing the middle of the Pacific can possibly run in.)
3. Look at ``fig04_shared_days_hist.png`` and End-notes B. We lose most possible station pairs to insufficient temporal overlap. Is the 90-day minimum the right threshold? What would you need to measure to answer that properly rather than guess?
4. Is the delivered network good enough to do science with yet, or do we wait? Argue it either way, but argue it with the numbers.

Write it up in the stage doc, then post the Slack summary. If something in the figures looks wrong to you, say so – "this looks wrong and here's why" is a completely valid result and I would rather hear it than have you assume I got it right.

TWO RULES, DO NOT BREAK THESE

1. Never read a channel dataset out of these HDF5 files. One channel is about 1.2 billion numbers – roughly 4.9 GB – and pulling one will hang or crash the node. The script only ever reads the tiny ``_coverage`` bitmaps, which is why the whole scan takes two minutes instead of a day. If you go exploring the files by hand, remember this.
2. The hard-link rule from the last memo still applies: the carried-forward ``h5`` files are hard links shared between v1 and v2. This entire analysis is read-only and must stay that way. Never write into ``packaged_h5/``.

And do not run any of this on terravibranium. It is pinned at a load of 42-44 on 48 cores running the SAmericaNoise packaging — it has nothing spare. The scan runs on BH3, the analysis runs on your own machine.

END-NOTES

A. What the delivered network actually looks like (my reference run, 2026-10-01)

Footprint

manifest stations	1,999
delivered (packaged, with data)	1,023 (51.2%)
days of real data	510,501
packaged on disk	502.1 GB
median days per station	229

Channel bands delivered: BH 767, LH 155, HH 80, EH 15, SH 5, MH 1. The EH/SH/MH stations are direct evidence the channel-list fix from the last memo worked – under the old `BH?,LH?` config those 21 stations would have returned nothing at all.

Ray paths (all separations, gated only on concurrent recording days)

usable station pairs	7,575
median shared days per pair	493
median path length	7,752 km

Illumination (3,000 equal-area cells, 412 km across)

cells with at least one path	3,000 (100%)
cells "resolved"	1,490 (49.7%)
median azimuthal spread A	0.607 (aspect ratio kappa = 2.30)
median effective resolution	831 km
unevenness of coverage (Gini)	0.352

Reconciliation, which the script checks every run: 1,999 manifest / 1,933 reported / 1,023 with data / 1,080 `.h5` files on disk. That last number being bigger is NOT a problem – those 57 extra files are stations currently being written. The result JSON only gets written when a station finishes, so a station in progress has a growing file and no JSON yet. Importantly, zero stations claim success without having their file on disk, which is the case that would mean real data loss.

B. The real constraint is time, not geometry — and it is temporary

With 1,023 stations there are 522,753 possible pairs. Only 7,575 survive. The reason is that you can only cross-correlate two stations over days they were BOTH recording, and our gate requires 90 shared days under 1,500 km and 365 over it.

Median delivered duration is 229 days, and the median station's data spans under a year. Two different one-year deployments that didn't overlap in time give you nothing, no matter how close together they are.

But this is a snapshot of a campaign that is 96% through, and the remaining 4% is not a random sample:

delivered	n=976	median 624 advertised days	28% have 5+ years
still in flight	n= 53	median 3,917 advertised days	96% have 5+ years

The stations still being processed are overwhelmingly the long-running permanent ones (G.AIS, G.CRZF, G.PAF and similar – they're still going because they're decades of data each, 15–18 GB per station). Every one of those will pair with almost everything else, because a 20-year station overlaps every short deployment inside its window. So treat 7,575 as a floor that will rise substantially, not as the final answer. Re-running this analysis after the campaign completes is the obvious follow-up, and the whole thing is built to be re-run cheaply for exactly that reason.

C. The formulas, explained properly

I want you to understand these rather than just read numbers off a colour bar.

C.1 Concurrent recording days, S

For a pair of stations, the number of days both were recording. This is the single most important quality number for one ray path. An NCF is built by stacking daily cross-correlations, and signal-to-noise grows roughly as the square root of the number of days stacked – so a pair with 2,000 shared days is about 4.7x cleaner than one with 90. We compute S exactly, from the per-day coverage bitmaps in the files.

Note I deliberately did NOT reduce S to a single "quality score". I could normalise it against, say, one year, but how many days are actually enough depends on noise source distribution, station noise floor, distance and period band, and varies by close to a factor of ten between pairs. So instead the script reports the full distribution and counts cells at several standards (90/180/365/730 days). You apply whichever standard you can defend; I'm not hiding a constant inside the number. Question 3 in your task is about exactly this.

C.2 Path count per cell, n

How many ray paths cross a given map cell. More is better, straightforwardly – more independent observations, better averaging, and room to throw out bad data without losing the cell. We want high redundancy, not low.

C.3 Azimuthal spread, A – this is the one worth understanding

The question: do the paths crossing a cell come from many directions, or are they all roughly parallel? A cell crossed by 50 parallel paths is badly resolved – you learn the velocity along one direction and nothing across it.

Why you can't just average the azimuths: angles wrap around. 359 degrees and 1 degree are 2 degrees apart, but their average is 180 – pointing the opposite way. Averaging angles directly is meaningless.

The standard fix: treat each azimuth as an arrow of length 1 on a circle, and add the arrows nose-to-tail. All pointing the same way, they stack into a long total. Spread evenly, they cancel into a short one. Divide by how many you added and you get a number between 0 and 1. That's R. (The notation $e^{i\theta}$ is just shorthand for "the arrow pointing at angle θ ".)

The seismology-specific catch: a ray path has no direction. A path running northeast at 45 degrees is the SAME LINE as one running southwest at 225. With raw azimuths those two arrows point exactly opposite and cancel – so two identical parallel paths would score as "perfectly spread", which is backwards. The fix is to DOUBLE every angle before adding: 45 becomes 90, and 225 becomes 450, which is the same as 90. Now they coincide and reinforce, correctly.

We also weight each arrow by how far that path travels inside the cell, since a path clipping a corner tells you less than one crossing the middle. Altogether:

$$R = | \text{sum over paths of } (L * \text{arrow at } 2*\theta) | / (\text{sum of } L)$$
$$A = 1 - R$$

so that bigger A = better, which is easier to read on a map.

Worked examples – the third one is why the doubling is there:

two paths both at 30 deg	doubled: 60, 60	R=1.0	A=0.0	parallel, worst case
--------------------------	-----------------	-------	-------	----------------------

one at 0, one at 90 deg	doubled: 0, 180	R=0.0	A=1.0	perpendicular, ideal
one at 0, one at 180 deg	doubled: 0, 360 = 0	R=1.0	A=0.0	same line, correctly scored as parallel
many, evenly spread	spread around	R~0	A~1	well sampled

C.4 Resolution aspect ratio, kappa

A converts directly into the shape of the resolution "blob" for that cell:

$$\text{kappa} = (1 + R) / (1 - R) = 2/A - 1$$

kappa = 1 is a circular blob – equally well resolved in every direction. kappa = 2.3 (our median) means we resolve structure about 2.3 times better along one direction than across it. This is the number that lets us make a defensible claim about what resolution our data actually supports, rather than just asserting one.

C.5 Effective resolution (Fresnel width)

A ray path isn't infinitely thin – it senses a zone around itself whose width grows as the square root of path length, $w = \sqrt{\lambda * L / 2}$. Longer paths smear structure over a wider zone. Taking the median over the paths crossing each cell gives an effective resolution in kilometres, which you can compare directly against the 412 km cell size. This is why allowing very long paths lights up the whole globe but at coarse resolution – it's the trade-off in fig15.

C.6 Unevenness (Gini) and redundancy (rho)

Gini measures how unevenly the paths are spread across cells; 0 would be perfectly even, ours is 0.352, which is moderately even. rho flags cells where many paths pile into few orientations. Both are descriptive – I'm not proposing we throw any data away.

D. Delivered vs planned, and why station count is the wrong headline

The obvious statement is "we have 51.2% of our stations". That badly misleads, because the number of station PAIRS goes roughly as the SQUARE of how densely stations are packed. If we lost half our stations at random we'd expect to keep only about a quarter of our pairs.

So the number I actually care about is

$$\text{psi} = (\text{fraction of pairs we kept}) / (\text{fraction of stations we kept, squared})$$

which tells you WHERE the losses fell:

- psi about 1 – losses were spread randomly
- psi below 1 – losses were concentrated in already-crowded regions. Cheap: those pairs were mostly redundant with pairs we still have.
- psi above 1 – losses were concentrated in sparse regions. Expensive: those pairs were the only ones crossing their part of the map.

We measured psi = 0.48, comfortably below 1. Our losses landed in dense regions. That is the good outcome and it's worth knowing, because it means chasing the remaining failed stations is lower priority than finishing the long-duration ones already in flight.

E. Two things I got wrong first, so you don't repeat them

Worth recording because both produced convincing-looking output that was simply false.

First, the job wouldn't start at all – it died inside ``conda activate`` complaining about ``GDAL_DATA: unbound variable``. The cause was ``set -u`` in my own job script, which makes bash abort on any undefined variable; the conda environment's own startup script reads that variable before setting it. Nothing to do with the analysis. The job script now says why, so don't add ``set -u`` back.

Second, and more instructive: my first map had beautiful curved stripes across it, which I briefly took for real structure. It wasn't. I was dividing the globe into 12,000 cells and assigning each point along a path to its nearest cell centre; because the cell centres sit on a spiral lattice, near-straight paths were snapping onto lattice rows and manufacturing the pattern. The giveaway was that the stripes followed the lattice rather than any geography. Fix was coarser cells (3,000) and finer sampling along each path. The deeper point is a real finding: with only 7,575 paths we cannot support a 206 km grid, even though the physics would allow it. We'll be able to once the long stations land.

F. Backup status

terravibranium's mirror is now complete for all finished stations: 1,023 files, 468 GB, matching BH3 exactly. I copied the 9 that were missing (BC.SPX, G.SEY, GE.PSZ, II.FFC, II.LVZ, II.SHEL, IU.KBS, NN.RUB, PQ.NBC8 – 16 GB). Nothing was deleted anywhere.

Still open, unchanged from the last memo: the mirror scripts are one-time snapshots and need re-running as more stations complete. Not yet automated.

G. Handoff to Tejwaswini

The analysis writes ``ncf_candidate_pairs.csv`` – the best-connected, longest-overlap station pairs, spread across distance ranges. That's deliberately the input Tejwaswini needs for the NCF step: rather than picking a pair arbitrarily, start from pairs we've measured as having the most concurrent data. It's also what we'd need to answer question 3 properly – compute NCFs at 30, 60, 90, 180, 365, 730 days of stacking and measure where signal-to-noise actually stops improving. That would replace the 90-day convention with a number measured from our own data.